Real-world (build and search a real knowledge base) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `knowledge_base_search_practice.py`: writing every document yourself means you're your own ground truth — when a search comes back wrong, you know it's wrong immediately, instead of having to guess whether the retrieval or the answer is at fault. **If not:** the Build Task's knowledge base would be the first place you ever had to debug a retrieval bug without knowing what the right answer was supposed to be.

Every example below assumes the same 6 sample documents (`docs/*.txt`):

```
# knowledge_base_search_practice.py
DOCS: dict[str, str] = {
    "French press basics.txt": (
        "A French press steeps coarse coffee grounds directly in hot "
        "water, then a mesh plunger separates the grounds. Steep for "
        "about 4 minutes before pressing. It produces a fuller-bodied, "
        "slightly gritty cup compared to filtered methods."
    ),
    "Pour-over basics.txt": (
        "Pour-over brewing means pouring hot water over grounds in a "
        "paper filter, in slow circular motions, letting it drip through "
        "into a cup or carafe below. It produces a clean, light-bodied "
        "cup because the paper filter traps most oils and fine particles."
    ),
    "Grind size guide.txt": (
        "Grind size should match your brew method. Use a coarse grind "
        "for French press, a medium grind for pour-over and drip "
        "machines, and a fine grind for espresso. Too fine a grind for "
        "the method causes over-extraction and bitterness."
    ),
    "Water temperature.txt": (
        "Water for brewing coffee should be between 195 and 205 degrees "
        "Fahrenheit, just off a full boil. Water that's too cool "
        "under-extracts and tastes sour or weak. Water that's too hot "
        "can scald the grounds and taste bitter."
    ),
    "Coffee-to-water ratio.txt": (
        "A common starting ratio is 1 gram of coffee to 16 grams of "
```

```
        "water, sometimes written as 1:16. For a stronger cup, move "
        "toward 1:14. For a milder cup, move toward 1:18. Adjust from "
        "there based on taste."
    ),
    "Storing coffee beans.txt": (
        "Store coffee beans in an airtight container, away from light "
        "and heat. Whole beans stay fresh for a few weeks after "
        "roasting; ground coffee stales much faster, within days. Don't "
        "store beans in the freezer for everyday use, since condensation "
        "on thawing hurts flavor."
    ),
}
```

Basic Version

Approach 1 — raw `chromadb` client

```
# knowledge_base_search_practice.py
from pathlib import Path
import chromadb
from openai import OpenAI

openai_client = OpenAI()

docs_folder = Path("docs")
docs_folder.mkdir(exist_ok=True)
for filename, text in DOCS.items():
    (docs_folder / filename).write_text(text)

def embed(text):
    response = openai_client.embeddings.create(
        model="text-embedding-3-small", input=text,
    )
    return response.data[0].embedding

def build_vector_store(doc_folder):
    chroma_client = chromadb.Client()
    collection = chroma_client.create_collection(name="knowledge_base")
    for file_path in Path(doc_folder).iterdir():
        text = file_path.read_text()
        embedding = embed(text)
        collection.add(
            documents=[text], embeddings=[embedding], ids=[file_path.name],
        )
    return collection
```

```
collection = build_vector_store("docs")

questions = [
    "What grind size should I use for a French press?",
    "How hot should the water be when brewing?",
    "How long does ground coffee stay fresh?",
]

for question in questions:
    question_embedding = embed(question)
    results = collection.query(
        query_embeddings=[question_embedding], n_results=3,
    )
    print("Q: " + question)
    print(results["ids"][0])
    print(results["documents"][0])
    print()
```

Expected output: since you wrote every doc yourself, question 1 should return `French press basics.txt` (and probably `Grind size guide.txt`) at the top, question 2 should return `Water temperature.txt`, and question 3 should return `Storing coffee beans.txt`. Read the printed results and check.

This version works correctly and meets every requirement of the exercise. Running the script a second time crashes, though — `create_collection` raises if `"knowledge_base"` already exists.

Approach 2 — only `.txt` / `.md` files, with the file name kept as metadata

Story: a real `docs/` folder also holds files that aren't documents — an editor's temp file, a `.DS_Store`, a PDF you meant to convert later. Approach 1's `iterdir()` embeds every one of them. And the search result only says *which text* matched, not *which file* it came from. **If not:** junk files would land in the store and come back as "answers", and a wrong result couldn't be traced back to its file.

This uses the plain `os` functions from Doc01: `os.listdir()` lists the folder, `.endswith()` checks the extension, `os.path.join()` builds the path, and `with open(...)` reads the file. Run Approach 1 once first, so `docs/` already holds the 6 files.

```
# knowledge_base_search_practice.py
import os
import chromadb
from openai import OpenAI

openai_client = OpenAI()
```

```
def embed(text):
    response = openai_client.embeddings.create(
        model="text-embedding-3-small", input=text,
    )
    return response.data[0].embedding


def build_vector_store(doc_folder):
    chroma_client = chromadb.Client()
    collection = chroma_client.create_collection(name="knowledge_base")
    for file_name in os.listdir(doc_folder):
        if not (file_name.endswith(".txt") or file_name.endswith(".md")):
            continue
        path = os.path.join(doc_folder, file_name)
        with open(path, "r") as f:
            text = f.read()
        collection.add(
            documents=[text],
            embeddings=[embed(text)],
            ids=[file_name],
            metadatas=[{"source_file": file_name}],
        )
    return collection


# a file that is not a document, like the ones editors leave behind
with open(os.path.join("docs", "notes.tmp"), "w") as f:
    f.write("scratch notes, not a document")

collection = build_vector_store("docs")
print("documents stored:", collection.count())

question = "How hot should the water be when brewing?"
results = collection.query(
    query_embeddings=[embed(question)], n_results=2,
)
metadatas = results["metadatas"][0]
texts = results["documents"][0]
for i in range(len(texts)):
    print(metadatas[i]["source_file"], "-", texts[i][:40])
```

Expected output (the second line can differ for you):

```
documents stored: 6
Water temperature.txt - Water for brewing coffee should be betwee
```

```
 Pour-over basics.txt - Pour-over brewing means pouring hot wate
```

`notes.tmp` was skipped, so the store holds the 6 real documents, not 7. Each result now carries its `source_file` in `results["metadatas"][0]`, in the same order as `results["documents"][0]` — so index `i` in one list always matches index `i` in the other.

Difference from Approach 1: Approach 1 embeds every file in the folder and returns only the text. Approach 2 keeps only `.txt` and `.md` files and stores each file's name as metadata, so every search result says where it came from.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — raw `chromadb`, factored into functions with type hints

```
# knowledge_base_search_practice.py
from pathlib import Path
import chromadb
from openai import OpenAI

openai_client = OpenAI()


def write_sample_docs(doc_folder: str, docs: dict[str, str]) -> None:
    folder_path = Path(doc_folder)
    folder_path.mkdir(exist_ok=True)
    for filename, text in docs.items():
        (folder_path / filename).write_text(text)


def embed(text: str) -> list[float]:
    response = openai_client.embeddings.create(
        model="text-embedding-3-small", input=text,
    )
    return response.data[0].embedding


def build_vector_store(doc_folder: str):
    chroma_client = chromadb.Client()
    collection = chroma_client.create_collection(name="knowledge_base")
    for file_path in Path(doc_folder).iterdir():
        text = file_path.read_text()
        embedding = embed(text)
        collection.add(
            documents=[text], embeddings=[embedding], ids=[file_path.name],
        )
    return collection


def ask(collection, question: str) -> None:
    question_embedding = embed(question)
    results = collection.query(
```

```
        query_embeddings=[question_embedding], n_results=3
    )
    print(f"Q: {question}")
    print(results["ids"][0])
    print(results["documents"][0])
    print()


def main() -> None:
    write_sample_docs("docs", DOCS)
    collection = build_vector_store("docs")
    questions = [
        "What grind size should I use for a French press?",
        "How hot should the water be when brewing?",
        "How long does ground coffee stay fresh?",
    ]
    for question in questions:
        ask(collection, question)


if __name__ == "__main__":
    main()
```

Approach 2 — LangChain's `Chroma` wrapper

```
# knowledge_base_search_practice.py
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings


def build_vector_store(docs: dict[str, str]) -> Chroma:
    texts = list(docs.values())
    metadatas = []
    for filename in docs.keys():
        metadatas.append({"source": filename})
    return Chroma.from_texts(texts, OpenAIEmbeddings(), metadatas=metadatas)


def ask(vector_store: Chroma, question: str) -> None:
    results = vector_store.similarity_search(question, k=3)
    print(f"Q: {question}")
    for result in results:
        print(result.metadata["source"], "-", result.page_content)
    print()
```

```
def main() -> None:
    vector_store = build_vector_store(DOCS)
    questions = [
        "What grind size should I use for a French press?",
        "How hot should the water be when brewing?",
        "How long does ground coffee stay fresh?",
    ]
    for question in questions:
        ask(vector_store, question)


if __name__ == "__main__":
    main()
```

Expected output: same 3 correct top matches as Basic. `Chroma.from_texts` embeds every doc for you, and `similarity_search` embeds the question for you — no manual `embeddings.create()` call anywhere.

Difference from Basic: full type hints, and everything split into named functions (`write_sample_docs`, `build_vector_store`, `ask`, `main`) instead of one flat script — each function does one job and can be tested or reused on its own. Approach 2 additionally removes the manual embedding calls entirely by letting LangChain's `Chroma` wrapper handle them. Neither approach yet survives being run a second time, or tells you when a question's best match genuinely isn't relevant — that's what the approaches below add.

Approach 3 — `get_or_create_collection`, safe to call more than once

Story: `create_collection(name="knowledge_base")` raises if `"knowledge_base"` already exists — run this script a second time, for any reason, and it crashes on line 2 before doing anything useful. **If not:** the Build Task's ingest step would be a one-shot script instead of something safe to re-run after adding or editing a document, which is the normal case for a real knowledge base.

```
# knowledge_base_search_practice.py
from pathlib import Path
import chromadb
from openai import OpenAI

openai_client = OpenAI()


def embed(text: str) -> list[float]:
    response = openai_client.embeddings.create(
        model="text-embedding-3-small", input=text,
    )
    return response.data[0].embedding
```

```
def build_vector_store(doc_folder: str):
    chroma_client = chromadb.Client()
    collection = chroma_client.get_or_create_collection(name="knowledge_base")

    file_paths = list(Path(doc_folder).iterdir())
    if collection.count() >= len(file_paths):
        # why: already built by an earlier call -- don't re-embed everything
        return collection

    for file_path in file_paths:
        text = file_path.read_text()
        embedding = embed(text)
        collection.add(
            documents=[text], embeddings=[embedding], ids=[file_path.name],
        )
    return collection


# proof this is now safe to call twice
first = build_vector_store("docs")
second = build_vector_store("docs")
print(first.count(), second.count())
```

Expected output:

```
6 6
```

The second call doesn't crash, and doesn't double-add every document either — `collection.count() >= len(file_paths)` short-circuits it.

Approach 4 — a score cutoff, so "no relevant match" is a real, checked outcome

Story: a nearest-neighbor search always returns *something* — ask an off-topic question and `collection.query()` still confidently hands back its `n_results` closest matches, with nothing signaling "none of these are actually good." **If not:** the Build Task's agent would silently receive 3 irrelevant chunks for an off-topic question and guess an answer from them, instead of being able to say "I don't have information on that."

```
# knowledge_base_search_practice.py
SCORE_CUTOFF: float = 1.2   # tune against your own embedding model + documents


def search(
```

```
    collection, question: str, k: int = 3, cutoff: float = SCORE_CUTOFF,
) -> list[dict]:
    question_embedding = embed(question)
    results = collection.query(
        query_embeddings=[question_embedding], n_results=k
    )

    matches: list[dict] = []
    for doc_id, text, distance in zip(
        results["ids"][0], results["documents"][0], results["distances"][0]
    ):
        if distance <= cutoff:
            matches.append({"id": doc_id, "text": text, "score": distance})

    return matches   # empty list means "nothing relevant," not "search failed"


collection = build_vector_store("docs")

good_question = "What grind size should I use for a French press?"
off_topic_question = "What's the best way to train a marathon?"

print("on-topic:", search(collection, good_question))
print("off-topic:", search(collection, off_topic_question))
```

Expected output (exact scores vary, but the shape holds):

```
on-topic: [{'id': 'Grind size guide.txt', 'text': '...', 'score': 0.62}, ...]
off-topic: []
```

What the score means: Chroma's default score is a *distance* — lower means closer. OpenAI embeddings have length 1, and for those Chroma's distance works out to `2 - 2 × cosine similarity`. So `0` means the same meaning, a good match with `text-embedding-3-small` usually lands around `0.6` – `1.0`, and an unrelated text lands around `1.5` – `1.8`. That's why the cutoff is `1.2`, between the two — and why you check it against your own good and off-topic questions before trusting it. The on-topic question returns real matches under the cutoff. The off-topic question still gets *a* nearest neighbor internally — nearest-neighbor search always finds something — but every distance is worse than `cutoff`, so `search()` filters all of them out and returns `[]` instead of confidently handing back 3 irrelevant chunks.

Approach 5 — `Path.glob()` and `.suffix`, and one batched embedding call

Story: Basic Approach 2's file filter, written the `pathlib` way, with type hints. It also embeds every document in **one** API call — the batching from the Basic exercise's Approach 2 — and adds them to

the collection in one `collection.add()` call. **If not:** a folder of 1,000 files would mean 1,000 separate embedding calls and 1,000 separate `add()` calls.

```
# knowledge_base_search_practice.py
from pathlib import Path
import chromadb
from openai import OpenAI

openai_client = OpenAI()

DOC_SUFFIXES = (".txt", ".md")


def load_documents(doc_folder: str) -> dict[str, str]:
    documents: dict[str, str] = {}
    for path in Path(doc_folder).glob("*"):
        if path.suffix not in DOC_SUFFIXES:
            continue
        documents[path.name] = path.read_text()
    return documents


def embed_batch(texts: list[str]) -> list[list[float]]:
    response = openai_client.embeddings.create(
        model="text-embedding-3-small", input=texts,
    )
    vectors: list[list[float]] = []
    for item in response.data:
        vectors.append(item.embedding)
    return vectors


def build_vector_store(doc_folder: str):
    documents = load_documents(doc_folder)
    file_names = list(documents.keys())
    texts = list(documents.values())
    metadatas: list[dict] = []
    for file_name in file_names:
        metadatas.append({"source_file": file_name})

    chroma_client = chromadb.Client()
    collection = chroma_client.create_collection(name="knowledge_base")
    collection.add(
        ids=file_names,
        embeddings=embed_batch(texts),
        documents=texts,
        metadatas=metadatas,
```

```
    )
    return collection


collection = build_vector_store("docs")
print("documents stored:", collection.count())
```

Expected output (with Basic Approach 2's `notes.tmp` still in `docs/`):

```
documents stored: 6
```

`path.suffix` is the extension with its dot (`".txt"`), so `not in DOC_SUFFIXES` skips `notes.tmp` . `path.name` is the file name without the folder, the same id as Basic Approach 2. The four lists passed to `collection.add()` must stay in the same order — `ids[i]` , `embeddings[i]` , `documents[i]` and `metadatas[i]` all describe one document.

Approach 6 — a clean rebuild: delete the collection, then create it again

Story: Approach 3 skips the work when the collection already holds enough documents — so after you **edit** a document, the old text stays in the store. Adding again does not help either: `collection.add()` with an id that already exists is silently ignored, and the old text is kept. **If not:** the knowledge base would keep answering from a document that no longer says that.

`delete_collection()` raises `NotFoundError` when the collection doesn't exist yet (the very first run). Catch exactly that error — Doc01's rule of catching the specific error you expect, not every error.

```
# knowledge_base_search_practice.py
import chromadb
from chromadb.errors import NotFoundError

chroma_client = chromadb.Client()


def rebuild_collection(name: str):
    try:
        chroma_client.delete_collection(name=name)
    except NotFoundError:
        pass  # first run: there was nothing to delete
    return chroma_client.create_collection(name=name)


def build_vector_store(doc_folder: str):
    documents = load_documents(doc_folder)   # from Approach 5
    file_names = list(documents.keys())
```

```
    texts = list(documents.values())
    metadatas: list[dict] = []
    for file_name in file_names:
        metadatas.append({"source_file": file_name})

    collection = rebuild_collection("knowledge_base")
    collection.add(
        ids=file_names,
        embeddings=embed_batch(texts),       # from Approach 5
        documents=texts,
        metadatas=metadatas,
    )
    return collection


build_vector_store("docs")

# edit one document, then build again
Path("docs/Water temperature.txt").write_text(
    "Use water at about 200 degrees Fahrenheit for every brew method."
)
collection = build_vector_store("docs")

stored = collection.get(ids=["Water temperature.txt"])
print("documents stored:", collection.count())
print(stored["documents"][0])
```

Expected output:

```
documents stored: 6
Use water at about 200 degrees Fahrenheit for every brew method.
```

The second build deleted the whole collection and made a new one, so the store holds the **new** text, and still 6 documents — nothing doubled. `collection.get(ids=[...])` reads stored documents by id, without a search. Always use the collection the rebuild **returns** — an older `collection` variable still points at the deleted one, and calling it raises `NotFoundError`. (Approach 1 writes the original text back the next time you run it.)

Approach 7 — LangChain's `Chroma`: scores, a folder on disk, and a clean rebuild

Story: Approach 2 used `similarity_search()`, which returns the documents but not how close they are, and kept the store only in memory. **If not:** a LangChain store could never use Approach 4's cutoff, and every run would re-embed every document.

`OpenAIEmbeddings(model=...)` names the embedding model, the same one as in every approach above. `collection_name` names the collection, like `name=` in raw `chromadb`. `persist_directory` saves the store in a folder on disk. `store.delete_collection()` empties it before a rebuild, the same idea as Approach 6. `similarity_search_with_score()` returns `(Document, score)` pairs — the score is the same distance as Approach 4 (lower = closer).

```
# knowledge_base_search_practice.py
from pathlib import Path
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings

EMBEDDINGS = OpenAIEmbeddings(model="text-embedding-3-small")


def build_vector_store(
    doc_folder: str, persist_directory: str = "./chroma_db",
) -> Chroma:
    old_store = Chroma(
        collection_name="knowledge_base",
        embedding_function=EMBEDDINGS,
        persist_directory=persist_directory,
    )
    old_store.delete_collection()

    texts: list[str] = []
    metadatas: list[dict] = []
    for path in Path(doc_folder).glob("*"):
        if path.suffix not in (".txt", ".md"):
            continue
        texts.append(path.read_text())
        metadatas.append({"source_file": path.name})

    return Chroma.from_texts(
        texts, EMBEDDINGS,
        metadatas=metadatas,
        collection_name="knowledge_base",
        persist_directory=persist_directory,
    )


def ask(store: Chroma, question: str, k: int = 3) -> None:
    print(f"Q: {question}")
    results = store.similarity_search_with_score(question, k=k)
    for document, score in results:
        print(f"  {score:.2f}  {document.metadata['source_file']}")
```

```
store = build_vector_store("docs")
ask(store, "What grind size should I use for a French press?")

empty_store = Chroma(
    embedding_function=EMBEDDINGS, persist_directory="./empty_db",
)
print("empty store:", empty_store.similarity_search_with_score("hi", k=3))
```

Expected output (your scores will differ; the order is what matters):

```
Q: What grind size should I use for a French press?
  0.62  Grind size guide.txt
  0.71  French press basics.txt
  0.98  Pour-over basics.txt
empty store: []
```

The store now lives in `./chroma_db/`, so a later script can open it with the same `collection_name`, `embedding_function` and `persist_directory` without embedding anything again. A search on an empty store returns `[]` instead of crashing.

Approach 8 — load and split with the library (`DirectoryLoader` + `RecursiveCharacterTextSplitter`)

Story: every approach so far read files with plain `open()`. That's fine for `.txt` and `.md`, but a real knowledge base also has PDFs, and PDFs aren't plain text. LangChain's **document loaders** read any of these into `Document` objects, and its **text splitter** cuts them into chunks that still remember which file (and page) they came from — the Document loaders topic. **If not:** the moment someone drops a PDF in `docs/`, your `open().read()` either crashes or stores raw binary.

This is the same **load → split → embed** pipeline, done the library way. `DirectoryLoader` loads the whole folder; `split_documents` keeps each chunk's `.metadata`; then it goes into the same `Chroma` store from Approach 7.

```
# knowledge_base_search_practice.py
from langchain_chroma import Chroma
from langchain_community.document_loaders import DirectoryLoader, TextLoader
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter

EMBEDDINGS = OpenAIEmbeddings(model="text-embedding-3-small")


def build_store_from_folder(folder: str):
    loader = DirectoryLoader(folder, glob="**/*.md", loader_cls=TextLoader)
```

```
    documents = loader.load()                      # one Document per file
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=500, chunk_overlap=50,
    )
    chunks = splitter.split_documents(documents)  # keeps each chunk's metadata
    return Chroma.from_documents(
        chunks, EMBEDDINGS, collection_name="knowledge_base",
    )


def ask(store, question: str, k: int = 3) -> None:
    print(f"Q: {question}")
    for document, score in store.similarity_search_with_score(question, k=k):
        print(f"  {score:.2f}  {document.metadata['source']}")


store = build_store_from_folder("docs")
ask(store, "What grind size should I use for a French press?")
```

To also read PDFs, swap `glob="**/*.pdf"` and `loader_cls=PyPDFLoader` (install `pypdf`) — the rest of the pipeline is unchanged, which is the whole point of loaders: the source format changes, your code doesn't.

Difference from Approach 7: Approach 7 wrote the sample files and embedded their whole text. Approach 8 *loads* a real folder with a `DirectoryLoader` and *splits* it with a text splitter before embedding — the real-world ingest path, and the only one that also handles PDFs.

Difference between Approaches 1–8: Approaches 1/2 work, but crash on a second call, and always return *something*, whether or not it's relevant. Approach 3 makes building safe to call twice with `get_or_create_collection` plus a cheap "already built" check — the same "check first, only do the real work if needed" pattern as Doc01's `load_config()` cache. Approach 4 makes "nothing here is relevant" a real, testable outcome. Approach 5 reads only real documents (`glob` + `suffix`) and embeds them in one batched call. Approach 6 replaces Approach 3's "skip if built" with a clean rebuild, so an edited document is picked up. Approach 7 does the same with LangChain's `Chroma`, and adds scores and a store saved on disk.

Which one should you actually use? Together, Approaches 4, 5 and 6 are what a real knowledge base needs. Read only real documents in one batched call (Approach 5), rebuild cleanly so edits are picked up (Approach 6 — use Approach 3's `get_or_create_collection` only when the documents never change), and judge every result against a cutoff you tuned on your own good and off-topic questions (Approach 4). Use Approach 7 when the rest of your code already uses LangChain. All of them feed the Build Task: its `ingest.py` is Approach 5 + 6 with chunks instead of

whole files, and its off-topic Test Case ("a bigger distance than an on-topic question") is Approach 4's distance check.